

Programação Visual e Autoria Web

Framework Angular

Universidade Federal do Rio Grande do Norte

Introdução

- Angular é um framework opensource desenvolvido pela Google para o desenvolvimento de clientes web utilizando TypeScript.
- Os blocos de construção básicos do Angular são os **Components** que eram organizados em **Modules** (ngModules).
- A partir do Angular 18+ novos projetos Angular já não usam **NgModules**
- **Standalone Components** passam a ser o padrão absoluto do framework

Standalone Components

- **Standalone Components** introduzem uma mudança de paradigma ao permitir que os **componentes sejam autossuficientes**, afastando-se da dependência de **NgModules**.
- **Atenção:** ao trabalhar com versões antigas de Angular você pode identificar a presença de NgModules. Nossas aulas vão se basear apenas em features recomendadas pela documentação atual.
- O decorador @Component já carrega, por padrão, as configurações para criação de Standalone Components.
- Os Standalone Components **encapsulam seus requisitos**, facilitando sua integração em toda a aplicação.
- Esses componentes são reconhecidos pelo **Angular como independentes**, eliminando a necessidade do código boilerplate associado aos NgModules nas versões modernas do Angular.

Standalone Components

- Standalone Components proporcionam **flexibilidade** além dos limites rígidos anteriormente estabelecidos pelos NgModules, que agrupavam componentes e suas dependências dentro de módulos.
- No paradigma antigo, **gerenciar dependências envolvia encapsular componentes, diretivas e pipes com importações e exportações em um único módulo.**
- O novo modelo sugere um processo simplificado onde os componentes incluem apenas as dependências que necessitam, manifestando uma **configuração modular e independente.**

Standalone Components

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('my-first-angular-20-app');
}
```

Standalone Components (app.html)

```
        </linearGradient>
        <clipPath id="a"><path fill="#fff" d="M0 0h982v239H0z"
/></clipPath>
    </defs>
</svg>
<h1>Hello, {{ title() }}</h1>
<p>Congratulations! Your app is running. 🎉</p>
</div>
<div class="divider" role="separator" aria-label="Divider"></div>
<div class="right-side">
```

Standalone Components (index.html)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>MyFistAngular20App</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

Components

- Cada aplicação Angular possui pelo menos um **Component**.
 - App component conecta uma hierarquia de componentes com o Document Object Model (DOM) da página.
- Cada **Component** define uma classe que contém dados e lógica da aplicação e está associada a um template HTML que define uma View a ser exibida.
- A anotação @Component() identifica a classe imediatamente abaixo dela como um **Component** e fornece o template e os metadados específicos do **Component** relacionado.

Components

- Um **Component** tem um ciclo de vida que começa quando o Angular instancia a classe do **Component** e renderiza a view do componente junto com as views filhas.
- O ciclo de vida continua com a detecção de alterações, pois o Angular verifica quando as propriedades vinculadas a dados são alteradas e atualiza a view e a instância do **Component** conforme necessário.
- O ciclo de vida termina quando o Angular destrói a instância do **Component** e remove seu modelo renderizado do DOM.

Exemplo de um component

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import Tarefa from '../models/Tarefa';
@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('my-fist-angular-20-app');
  protected readonly tarefas = [
    new Tarefa('Estudar', true),
    new Tarefa('Jogar', false),
    new Tarefa('Passear', false),
  ];
  protected readonly imagem =
    'https://www.ufrn.br/resources/documentos/identidadevisual/logotipo/logotipo_flat.png';
}
```

```
class Tarefa {
  public name: String;
  public priority: Boolean;

  constructor(name: String, priority:
Boolean) {
    this.name = name;
    this.priority = priority;
  }
}

export default Tarefa;
```

Exemplo de um Component

```
<main>
  <h1>{{ title() }}</h1>

  <ul>
    @for (t of tarefas; track $index) {
      <li>{{ t.name }}</li>
    }
  </ul>

  <img [src]="imagem" width="100%" />
</main>

<router-outlet />
```

Templates

- É possível definir a View de um **Component** a partir do seu template.
- Um template é uma “espécie de HTML” que informa ao Angular como renderizar o Component
- As Views geralmente são organizadas hierarquicamente, permitindo a modificação ou exibição/ocultação de seções ou páginas inteiras da interface do usuário em um único lugar.

Templates

- Um template se parece com um HTML normal, exceto que ele também contém a **sintaxe de templates do Angular** que altera o HTML com base na lógica da aplicação.
- O template usa **data binding** para coordenar a aplicação e DOM.
- Além disso, é possível utilizar **pipes para transformar os dados antes de serem exibidos** e diretivas para aplicar a lógica da aplicação ao que é exibido.

Templates

```
<h2>Hero List</h2>
<p><em>Select a hero from the list to see details.</em></p>
<ul>
  @for (let hero of heroes; track hero){
    <li>
      <button type="button" (click)="selectHero(hero)">
        {{ hero.name }}
      </button>
    </li>
  }
</ul>
```

- Exemplo, a diretiva @For diz ao Angular para iterar sobre uma lista
- {{hero.name}}, (click) e [hero] vinculam dados do programa com o DOM, respondendo à entrada do usuário.

Angular CLI

- Angular CLI é uma ferramenta de interface de linha de comando para inicializar, desenvolver, estruturar e manter aplicativos Angular diretamente de um terminal.
- Necessário instalar o Node.

```
npm install -g @angular/cli  
npx ng --help
```

```
ng new my-first-project  
cd my-first-project  
ng serve
```

Desenvolvimento de um projeto simples

```
npx ng new angular-aula-01
```

- *Which stylesheet format would you like to use? (CSS, SCSS, SCSS (indented), Less*
 - **CSS (Cascading Style Sheets)** é uma linguagem de estilo usada para descrever a apresentação de um documento escrito em HTML ou XML.
 - **SCSS (Sassy CSS)** é uma sintaxe para SASS (Syntactically Awesome Stylesheets), que é uma extensão do CSS que permite usar variáveis, aninhamento, mixins, entre outros.

Desenvolvimento de um projeto simples

```
npx ng new angular-aula-01
```

- *Which stylesheet format would you like to use? (CSS, SCSS, SCSS (indented), Less*
 - **SCSS (Indented Syntax)** é também conhecido como SASS (sem SCSS), é uma das duas sintaxes do SASS. Usa indentação em vez de chaves e ponto-e-vírgulas.
 - **LESS (Leaner CSS)** é uma extensão do CSS que adiciona funcionalidades como variáveis, aninhamento, mixins, etc.

Principais diferenças

- **CSS:** Puro e direto, sem variáveis ou aninhamento.
- **SCSS:** Similar ao CSS com adição de variáveis, aninhamento, mixins, etc.
- **SCSS (Indented):** Sintaxe baseada em indentação, sem chaves ou ponto-e-vírgulas.
- **LESS:** Similar ao SCSS em funcionalidade, com uma sintaxe própria.

Desenvolvimento de um projeto simples

CSS

```
body {  
  background-color: #f0f0f0;  
}  
h1 {  
  color: blue;  
}
```

SCSS

```
$primary-color: #333;  
  
body {  
  background-color: $primary-color;  
  font-family: Arial, sans-serif;  
  
  h1 {  
    color: blue;  
  }  
}
```

LESS

```
@primary-color: #333;  
  
body {  
  background-color: @primary-color;  
  font-family: Arial, sans-serif;  
  
  h1 {  
    color: blue;  
  }  
}
```

SCSS Indentado

```
$primary-color: #333  
  
body  
  background-color: $primary-color  
  font-family: Arial, sans-serif  
  
  h1  
    color: blue
```

Popularidade e Suporte

- **CSS:** Padrão universal suportado por todos os navegadores.
- **SCSS:** Amplamente adotado em muitos projetos modernos.
- **LESS:** Popular, mas menos utilizado que SCSS atualmente.
- **SCSS (Indented):** Menos utilizado devido à preferência por sintaxes mais similares ao CSS tradicional.

Desenvolvimento de um projeto simples

```
npx ng new angular-aula-01
```

- *Do you want to enable Server-Side Rendering (SSR) and Static Site Generation?*
 - Técnicas diferentes para gerar e servir páginas web, que podem melhorar o desempenho e a experiência do usuário em certos tipos de aplicações web
 - Respondendo não, vamos gerar uma **aplicação de página única padrão (single page application)**.

Desenvolvimento de um projeto simples

```
npx ng new angular-aula-01
```

- **Single Page Application:** Toda a aplicação é carregada no cliente, e uma única página é carregada. A aparência de navegação por toda a aplicação é controlada dinamicamente através do JavaScript no cliente.
- **Server Side Rendering:** Em vez de entregar a aplicação inteiramente para o cliente, a aplicação será executada em um servidor externo. Solicitações de páginas podem ser feitas para esse servidor, o servidor renderizará essa página e depois a entregará ao cliente.
 - Plataformas de e-commerce, dashboards, aplicativos web interativos.
- **Static Site Generation:** A aplicação é construída uma vez para criar muitas páginas estáticas diferentes, e então essas páginas são carregadas diretamente de um servidor (em vez de serem geradas pelo servidor dinamicamente).“
 - Blogs, sites de portfólio, documentação, landing pages.

Desenvolvimento de um projeto simples

```
npx ng new angular-aula-01
```

- *Do you want to create a 'zoneless' application without zone.js? (y/N)?*
 - Zone.js é uma biblioteca JavaScript usada pelo Angular para detectar automaticamente mudanças no estado da aplicação e disparar a detecção de mudanças (change detection) da UI.
 - Ela funciona como um “patch” em APIs assíncronas do JavaScript (como setTimeout, Promise, eventos do DOM, WebSockets etc.), criando um contexto de execução chamado zona, para que o Angular saiba quando algo assíncrono terminou e possa atualizar a tela.

Desenvolvimento de um projeto simples

```
npx ng new angular-aula-01
```

- *Do you want to create a 'zoneless' application without zone.js? (y/N)?*
 - Desde o Angular 17, com a introdução dos signal-based components e do novo sistema de change detection, o Angular não precisa mais do Zone.js para funcionar.
 - Você pode continuar usando Zone.js (modo compatibilidade)

Desenvolvimento de um projeto simples

The screenshot displays the Visual Studio Code interface for a project named 'my-fist-angular-20-app'. The Explorer sidebar on the left shows the project structure, including a 'src' directory with files like 'app.config.ts', 'app.css', 'app.html', 'app.routes.ts', 'app.spec.ts', 'app.ts', 'models', 'Tarefa.ts', 'index.html', and 'main.ts'. The 'main.ts' file is selected and open in the editor. The code in 'main.ts' is as follows:

```
src > TS main.ts > ...
1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { appConfig } from './app/app.config';
3 import { App } from './app/app';
4
5 bootstrapApplication(App, appConfig)
6   .catch((err) => console.error(err));
7
```

The bottom panel shows the TERMINAL view with the following output:

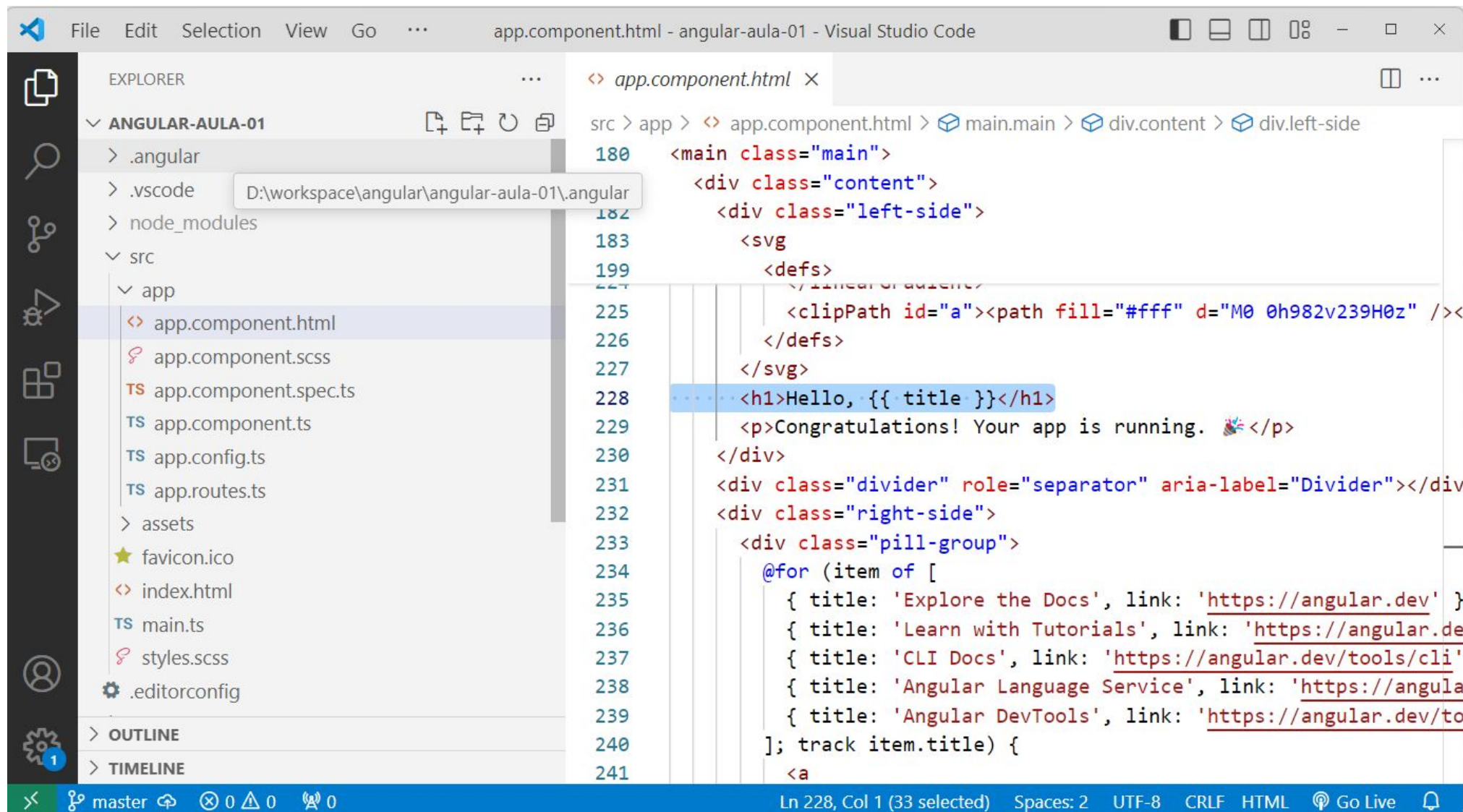
```
PS D:\angular\my-fist-angular-20-app> ng serve

Initial chunk files | Names | Raw size
main.js            | main | 3.92 kB |

Application bundle generation complete. [0.083 seconds] - 2025-11-10T15:26:52.307Z

Component update sent to client(s).
```

Desenvolvimento de um projeto simples

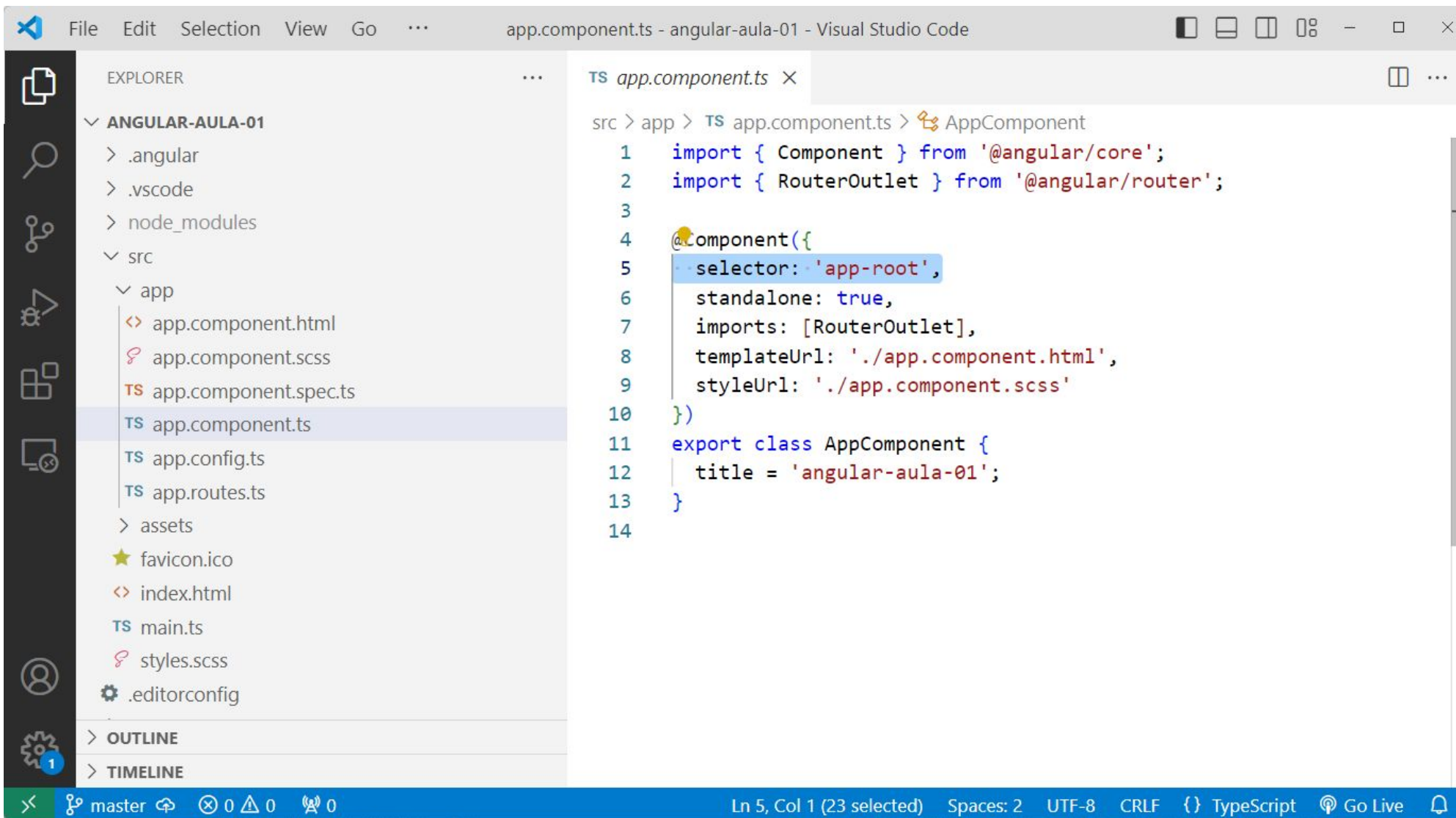


```
File Edit Selection View Go ... app.component.html - angular-aula-01 - Visual Studio Code

EXPLORER
ANGULAR-AULA-01
  .angular
  .vscode
  node_modules
  src
    app
      app.component.html
      app.component.scss
      app.component.spec.ts
      app.component.ts
      app.config.ts
      app.routes.ts
    assets
    favicon.ico
    index.html
    main.ts
    styles.scss
    .editorconfig
  OUTLINE
  TIMELINE

src > app > app.component.html > main.main > div.content > div.left-side
180 <main class="main">
182   <div class="content">
183     <div class="left-side">
199       <svg
225         <clipPath id="a"><path fill="#fff" d="M0 0h982v239H0z" /><
226       </clipPath>
227     </svg>
228     <h1>Hello, {{ title }}</h1>
229     <p>Congratulations! Your app is running. 🎉</p>
230   </div>
231   <div class="divider" role="separator" aria-label="Divider"></div>
232   <div class="right-side">
233     <div class="pill-group">
234       @for (item of [
235         { title: 'Explore the Docs', link: 'https://angular.dev' }
236         { title: 'Learn with Tutorials', link: 'https://angular.de
237         { title: 'CLI Docs', link: 'https://angular.dev/tools/cli'
238         { title: 'Angular Language Service', link: 'https://angula
239         { title: 'Angular DevTools', link: 'https://angular.dev/to
240       ]; track item.title) {
241         <a
```

Desenvolvimento de um projeto simples



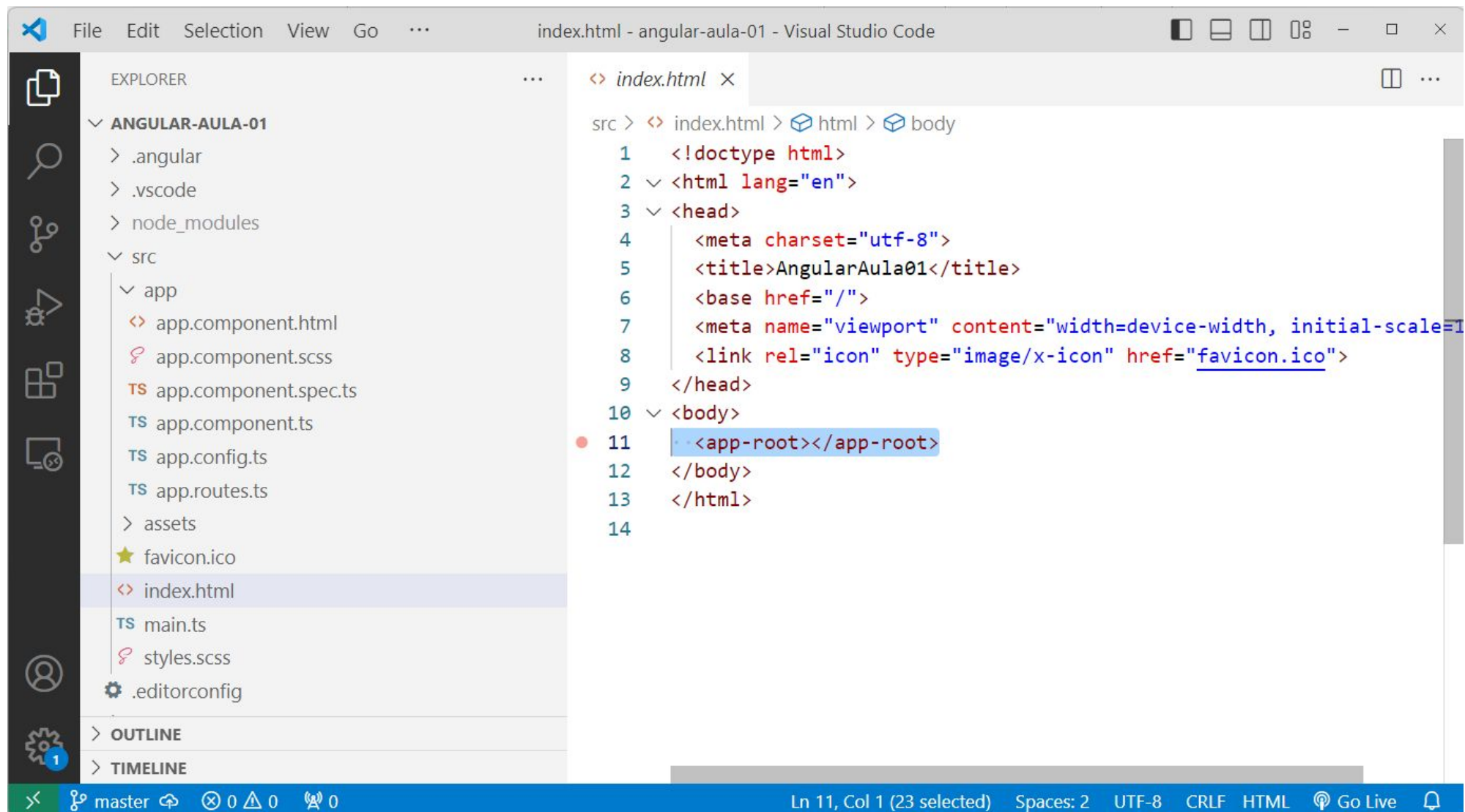
```
File Edit Selection View Go ... app.component.ts - angular-aula-01 - Visual Studio Code

EXPLORER
ANGULAR-AULA-01
  .angular
  .vscode
  node_modules
  src
    app
      app.component.html
      app.component.scss
      app.component.spec.ts
      app.component.ts
      app.config.ts
      app.routes.ts
    assets
    favicon.ico
    index.html
    main.ts
    styles.scss
    .editorconfig
  OUTLINE
  TIMELINE

src > app > TS app.component.ts > AppComponent
1  import { Component } from '@angular/core';
2  import { RouterOutlet } from '@angular/router';
3
4  @Component({
5    selector: 'app-root',
6    standalone: true,
7    imports: [RouterOutlet],
8    templateUrl: './app.component.html',
9    styleUrls: ['./app.component.scss']
10 })
11 export class AppComponent {
12   title = 'angular-aula-01';
13 }
14

Ln 5, Col 1 (23 selected) Spaces: 2 UTF-8 CRLF {} TypeScript Go Live
```

Desenvolvimento de um projeto simples



Aplicação to-do List

- Vamos desenvolver uma aplicação de To-Do List para treinar os primeiros conceitos de Angular.
- O primeiro passo é a criação da classe /shared/model/Tarefa

```
export class Tarefa {  
    public texto: string;  
    public feito: boolean;  
  
    constructor(texto: string, feito: boolean = false) {  
        this.texto = texto;  
        this.feito = feito;  
    }  
};
```


Aplicação to-do List

- Vamos desenvolver uma aplicação de To-Do List para treinar os primeiros conceitos de Angular.
- O primeiro passo é a criação da classe /shared/model/Tarefa

```
export class Tarefa {  
  constructor(public texto: string, public feito: boolean = false) {}  
}
```

Aplicação to-do List

- O próximo passo é a importação da classe Tarefa dentro do Component e a criação de algumas tarefas de exemplo.

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import Tarefa from '../models/Tarefa';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('my-fist-angular-20-app');
  protected readonly tarefas = [
    new Tarefa('Estudar', true),
    new Tarefa('Jogar', false),
    new Tarefa('Passear', false),
  ];
  protected readonly imagem = 'https://www.ufrn.br/resources/documentos/identidadevisual/logotipo/logotipo_flat.png';
}
```

Aplicação to-do List

- Adicionando bootstrap na aplicação, alterando o arquivo index.html.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>MyFistAngular20App</title>
    <base href="/" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="icon" type="image/x-icon" href="favicon.ico" />
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
      crossorigin="anonymous"
    />
  </head>
  <body>
    <app-root></app-root>
    <script
      src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
      integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
      crossorigin="anonymous"
    ></script>
  </body>
</html>
```

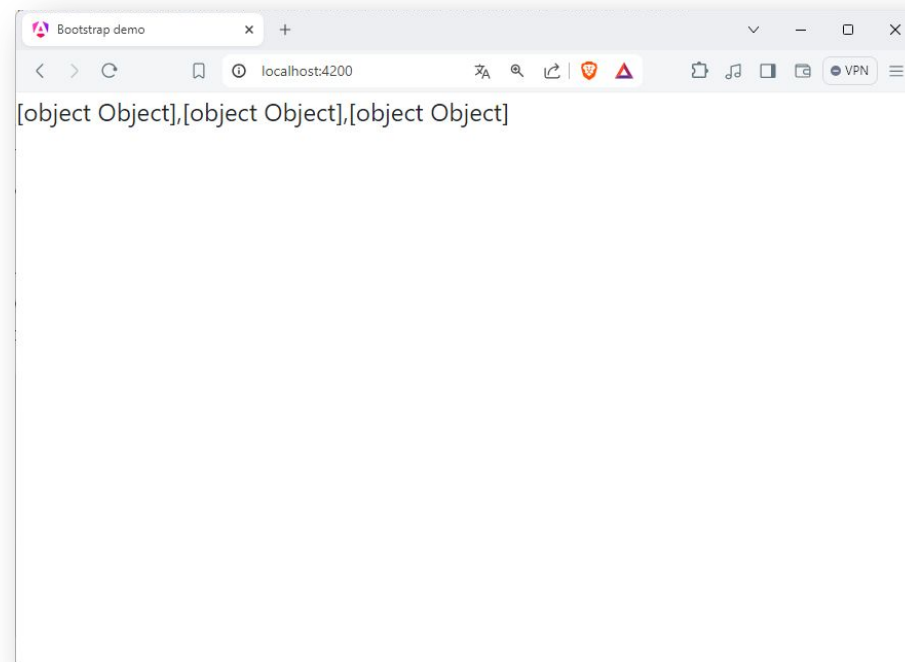

Templates - Interpolação

- Apresentando os dados das tarefas criadas no componente ao editar o arquivo **app.component.html**
- **Interpolação** refere-se à inserção de expressões em texto marcado. **Por padrão, a interpolação usa as chaves duplas {{ e }}** como delimitadores.



```
{{ tarefas }}
```

```
{{ tarefas[0].texto }}
```



Templates – Template Statements

- Template Statements são **métodos ou propriedades** que podem ser usadas no HTML para responder a eventos do usuário.
- Com Template Statements a aplicação pode interagir com os usuários através de ações como exibir conteúdo dinâmico ou enviar formulários.

```
<button type="button" (click)="deleteHero()">Delete hero</button>
```

- No exemplo, o Template Statement `deleteHero()` aparece entre aspas à direita do sinal de igual `=`, como em `(event)="statement"`.
- Quando o usuário clicar no botão "Delete hero", o Angular chama o método `deleteHero()` na classe do componente.

Templates – Template Statements

- Template Statements possuem um contexto
 - Ou seja, uma parte específica da aplicação à qual o statement pertence.
- Statements podem se referir apenas ao que está em seu contexto, que é tipicamente a instância do componente.
 - Por exemplo, `deleteHero()` de `(click)="deleteHero()"` é um método do componente.

Templates – Template Statements

- O contexto de um Template Statement também pode se referir a **propriedades do próprio contexto do template**.
- No exemplo o método onSave(), recebe o objeto **\$event** do próprio template como argumento. O método deleteHero() recebe uma **variável de entrada do template**, hero, e onSubmit() recebe uma variável de referência do template, #heroForm.

```
<button type="button" (click)="onSave($event)">Save</button>
...
@for (hero of heroes; track hero) {
  <button type="button" (click)="deleteHero(hero)">{{ hero.name }}</button>
}
...
<form #heroForm (ngSubmit)="onSubmit(heroForm)"> ... </form>
```

Templates - Binding

- Em Angular um binding (ligação) cria uma **conexão entre uma parte da interface** do usuário criada a partir de um template (um elemento DOM, diretiva ou componente) e o **modelo** (a instância do componente ao qual o template pertence).
- Essa conexão pode ser usada para sincronizar a visualização com o modelo, para notificar o modelo quando um evento ou ação do usuário ocorre na visualização, ou ambos.
- Exemplos de binding incluem:
 - Interpolação de texto
 - Binding de propriedade
 - Binding de evento
 - Binding bidirecional
- Bindings sempre têm duas partes: um alvo, que receberá o valor vinculado, e uma expressão de template, que produz um valor a partir do modelo.

Templates - Binding de propriedade

- A ligação de propriedade (property binding) no Angular ajuda a definir valores para propriedades de elementos HTML ou diretivas.
- O binding de propriedade é usado para realizar ações como alternar funcionalidades de botões, definir caminhos programaticamente e compartilhar valores entre componentes.
- O binding de propriedade move um valor em uma direção, de uma propriedade do componente para uma propriedade do elemento alvo.

Templates - Binding de propriedade

- Para vincular a uma propriedade de um elemento, coloque-a entre colchetes, [], o que identifica a propriedade como uma propriedade alvo.
- Uma propriedade alvo é a propriedade DOM à qual você deseja atribuir um valor.

```
export class AppComponent {  
  imagem = "https://www.metroledigital.ufrn.br/portal/img/ufrn-sem-legenda.svg";  
}
```

```
  
  
<img [src]="imagem" />
```

Templates - Binding de propriedade

- Os colchetes, [], fazem com que o Angular avalie o lado direito da atribuição como uma expressão dinâmica.
- Sem os colchetes, o Angular trata o lado direito como uma string literal e define a propriedade com esse valor estático.

```
export class AppComponent {  
  imagem = "https://www.metroledigital.ufrn.br/portal/img/ufrn-sem-legenda.svg";  
}
```

```
  
  
<img [src]="imagem" />
```


Templates - Binding de propriedade

- Casos especiais:
 - Calmel cases como colspan (atributo HTML) e colSpan (propriedade)
 - disabled em botões de formulário e [disabled] = “boolean”

Templates – Binding de Attribute

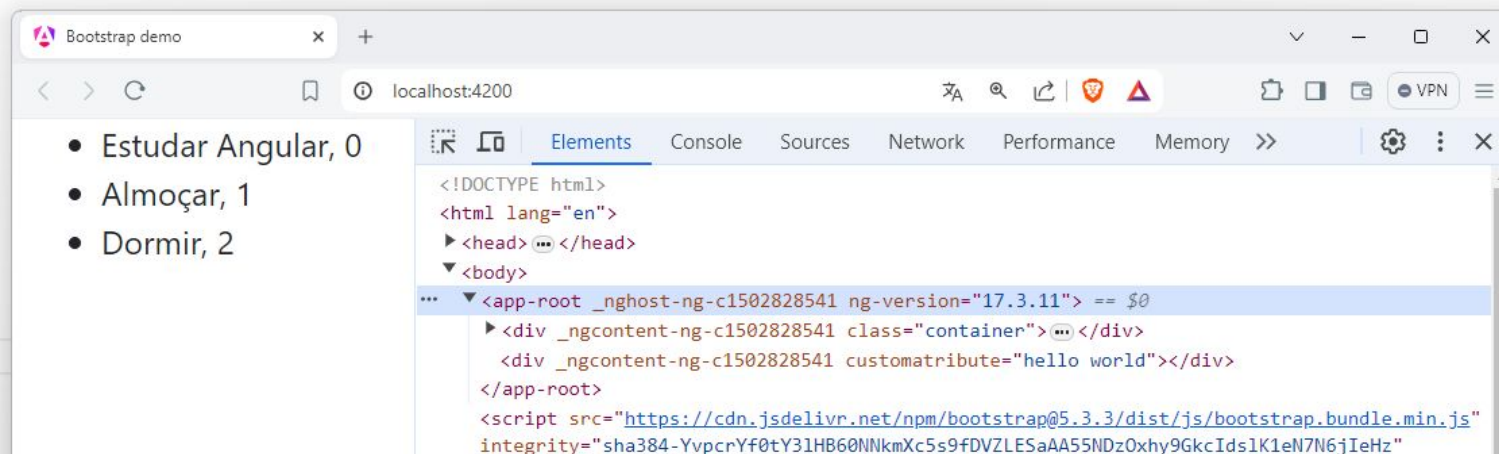
- O Binding de Attribute no Angular ajuda a definir valores para atributos diretamente.
 - Usado para melhorar a acessibilidade, estilizar a aplicação dinamicamente e gerenciar múltiplas classes ou estilos CSS simultaneamente.
- A sintaxe do Binding de Attribute se assemelha ao Binding de propriedade:
 - Ao invés de uma propriedade do elemento entre colchetes
 - O nome do atributo será precedido com attr., seguido de um ponto.
 - O valor do atributo deve receber uma expressão que resolve para uma string.

```
<p [attr.attribute-you-are-targeting]="expression"></p>
```

Templates – Binding de Attribute

src > app > <> app.component.html > div

```
Go to component
1 <div class="container">
2   <ul>
3     @for (item of tarefas; track $index) {
4       <li>{{item.texto}}, {{ $index }}</li>
5     }
6   </ul>
7 </div>
8
9 <div [attr.customattribute]="dadosAtributo"></div>
```



- Um dos principais casos de uso para Binding de Attribute é definir atributos ARIA (Accessible Rich Internet Applications).

```
<!-- create and set an aria attribute for assistive technology -->
<button type="button" [attr.aria-label]="actionName">{{ actionName }} with Aria</button>
```

Templates – Binding de Class e Style

- Para adicionar e remover nomes de classes CSS dinamicamente usando ligação de classe (class binding) e para definir estilos dinamicamente usando ligação de estilo (style binding) em um elemento

```
<div [class.error]="isError">Erro ocorreu!</div>
```

- A classe error será aplicada à <div> apenas se isError for verdadeiro no componente Angular.

Templates – Binding de Class e Style

```
<div [class]="classExpression">Elemento com classes dinâmicas</div>
```

- classExpression pode ser uma das seguintes:
 - Uma string com nomes de classes separados por espaços.
 - Um objeto onde os nomes das classes são as chaves e expressões verdadeiras ou falsas como os valores.
 - Um array de nomes de classes.

Templates – Binding de Class e Style

- Um estilo também pode ser definido dinamicamente.
- Para criar uma única Binding de estilo, use-se o prefixo `style` seguido de um ponto e o nome do estilo CSS.
 - `[style.width]="largura"`
- Angular atribui à propriedade o valor da expressão vinculada, que geralmente é uma string. Opcionalmente, você pode adicionar uma unidade como **em** ou **%**, o que requer um tipo numérico.

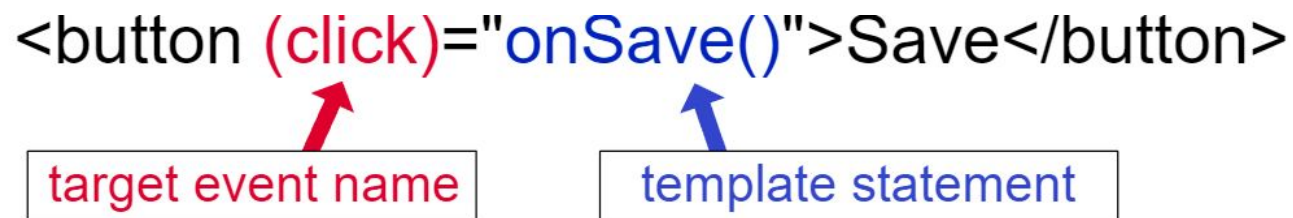
Templates – Binding de Class e Style

- Para alternar múltiplos estilos, vincule ao atributo [style] — por exemplo, [style]="expressãoDeEstilo". A expressãoDeEstilo pode ser uma das seguintes:
 - Uma lista de estilos em forma de string, como "width: 100px; height: 100px; background-color: cornflowerblue;".
 - Um objeto com nomes de estilos como chaves e valores de estilos como valores, como {width: '100px', height: '100px', backgroundColor: 'cornflowerblue'}.

Templates – Event Binding

- Event Binding permite que ouvir e responder a ações do usuário, como pressionamentos de tecla, movimentos do mouse, cliques e toques.
- A sintaxe de um Event Binding consiste no nome do evento alvo entre parênteses à esquerda de um sinal de igual, e uma Template Statement entre aspas à direita.

`<button (click)="onSave()">Save</button>`



target event name

template statement

Templates – Controw Flow

- Os templates do Angular suportam blocos de controle de fluxo que permitem exibir, ocultar e repetir elementos condicionalmente.
- Na versão 17 do angular podemos usar uma sintaxe simplificada de diretivas utilizando @If, @For, @Else, etc.
- Códigos anteriores vão possuir as respectivas diretivas ngIf, ngFor, etc.
- Vamos adotar a versão atual que simplifica nossos códigos.

@If

- O bloco @if exibe condicionalmente seu conteúdo quando sua expressão de condição é verdadeira.

```
@if (a > b) {  
  {{a}} is greater than {{b}}  
}
```

@If

- O bloco @if pode ter um ou mais blocos @else associados. Logo após um bloco @if, você pode opcionalmente especificar qualquer número de blocos @else if e um bloco @else.

```
@if (a > b) {  
  {{a}} is greater than {{b}}  
} @else if (b > a) {  
  {{a}} is less than {{b}}  
} @else {  
  {{a}} is equal to {{b}}  
}
```

@For

- O bloco @for renderiza repetidamente o conteúdo de um bloco para cada item em uma coleção.
- A coleção pode ser representada como qualquer iterável do JavaScript, mas existem vantagens de desempenho ao usar um Array.

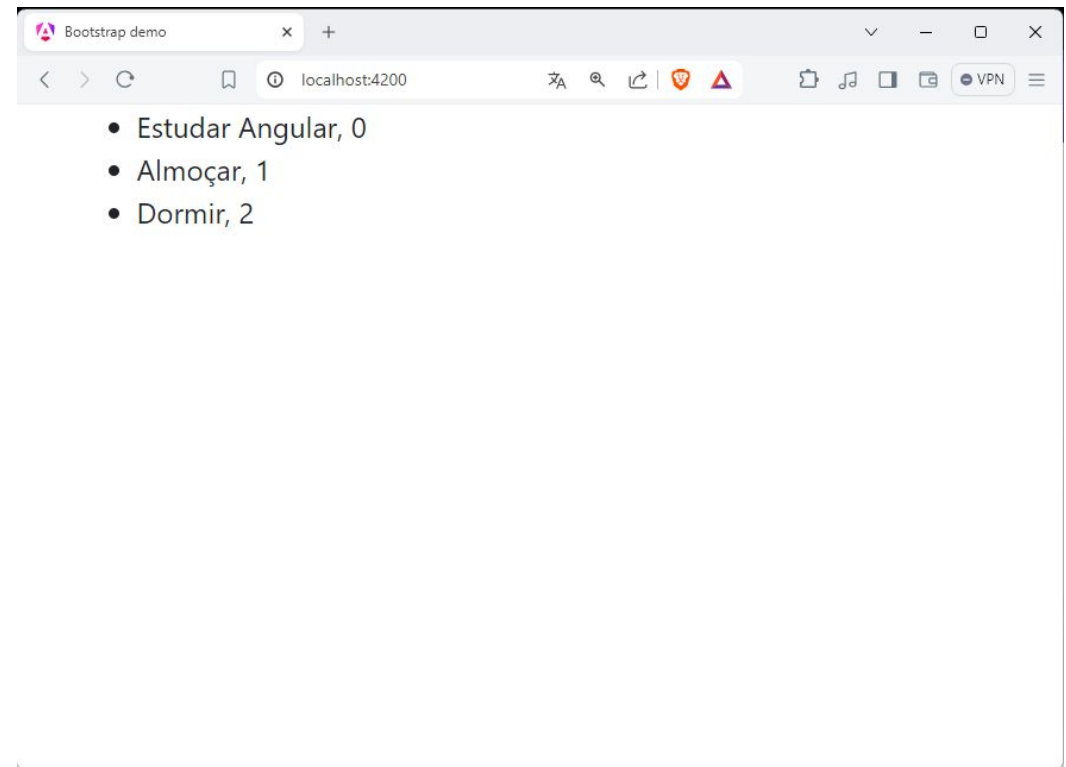
```
@for (item of items; track item.id) {  
  {{ item.name }}  
}
```

@For

- A declaração track item é usada para rastrear os itens por referência.
- Esta declaração é necessária.
- Ela é usada para otimizar o desempenho ao evitar execuções desnecessárias de detecção de mudanças quando os dados mudam.
- Se esta declaração não for incluída, você receberá um erro de compilação.

@For

```
<div class="container">
  <ul>
    @for (item of tarefas; track $index) {
      <li>{{item.texto}}, {{$index}}</li>
    }
  </ul>
</div>
```



@For

- Dentro do conteúdo do @for, várias variáveis implícitas estão sempre disponíveis:
 - \$count Número de itens na coleção iterada
 - \$index Índice da linha atual
 - \$first Indica se a linha atual é a primeira
 - \$last Indica se a linha atual é a última
 - \$even Indica se o índice da linha atual é par
 - \$odd Indica se o índice da linha atual é ímpar

@For

```
@for (item of items; track item.id; let idx = $index, e = $even) {  
  Item #{{ idx }}: {{ item.name }}  
}
```


@For – Empty block

- Você pode opcionalmente incluir uma seção @empty imediatamente após o conteúdo do bloco @for.
- O conteúdo do bloco @empty é exibido quando não há itens.

```
@for (item of items; track item.name) {  
  <li> {{ item.name }}</li>  
} @empty {  
  <li> There are no items.</li>  
}
```

@Switch

- A sintaxe para switch é muito semelhante ao if, e é inspirada na instrução switch do JavaScript.

```
@switch (condition) {  
  @case (caseA) {  
    Case A.  
  }  
  @case (caseB) {  
    Case B.  
  }  
  @default {  
    Default case.  
  }  
}
```

Aplicação to-do List

- Com base nos conhecimentos de templates vistos na aula desenvolva um componente para apresentar uma lista de tarefas.
- Cada tarefa deve ser acompanhada de um checkbox;
- Se a tarefa está com o atributo feita = true, o checkbox deve aparecer marcado e o nome da tarefa ~~taxado~~.
- Ao clicar em um checkbox para marcar/desmarcar o atributo feita deve ser atualizado e os efeitos visuais devem acompanhar.

Referências

- <https://material.angular.io/>
- <https://angular.io/>
- <https://borstch.com/blog/development/angulars-ngmodules-vs-standalone-components-pros-and-cons>